csesoc UNSW

Sydney Quant Traders

CSEsoc X SQT presents.....

C++ WORKSHOP:

Templates + More!

Date : Week 7
Wed 28 Oct 2026

Time: 15:00 – 17:00

Location: Ainsworth 202



ATC CLUBS
SUPPORTED BY ATC MANAGEMENT

Before we start

Prerequisites

- basic OOP in C++ (methods, initialization, constructors)
- basic assembly concepts (labels, stack frames)

Workshop topics

- Objects, RAII
- Move semantics, temporaries, RVO
- Templates, universal references



Conventions and Notation

- will use x86-64, System V ABI, -O0
- I will sometimes refer to methods, constructors and destructors by their full demangled function name e.g. `ExampleClassName::exampleMethodName(arg1, arg2)` .

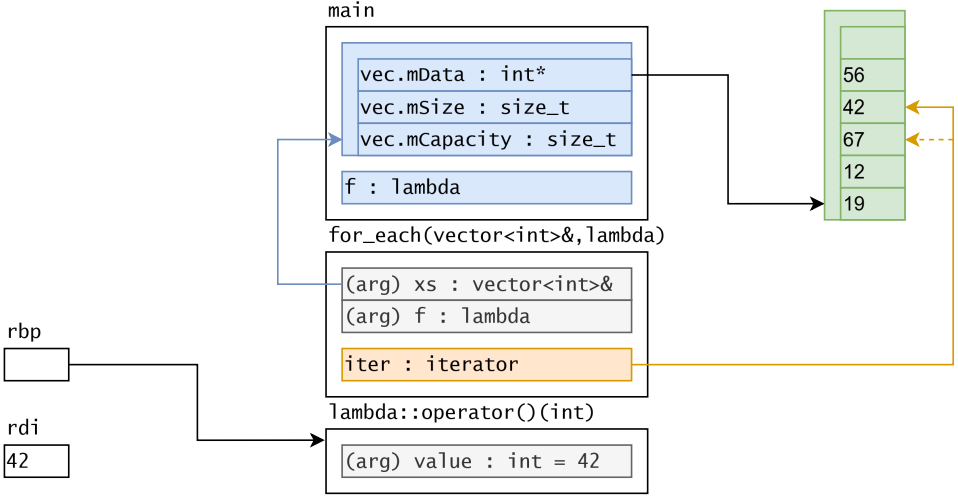


Visualizations

- We will sometimes visualize the state of a program at a snapshot in time.
- The stack will be drawn as growing downwards.



Visualizations



Object Model Revision

C++ object lifecycle:

- Objects can be constructed in some storage location (their lifetime begins).
- When a destructor is called at that storage location, the lifetime ends.

For stack allocated objects:

- Storage location is known at compile time (offset within stack frame).
- Lifetime is known at compile time (when the scope ends).
  - Lifetime begins when the constructor is called on the storage location.
  - Compiler generates calls to the destructor at the end of the scope.



Objects and Methods

A *class* (in a general, language-agnostic sense) is approximately just a schema describing what data is present in an object. and a collection of functions that read/mutate objects satisfying that schema.

The C++ *class* language feature lets you define the schema and the collection of functions ergonomically in one place.

Methods (including constructors and destructors) implicitly take the *address of the object* as an argument (binds to the argument `this`, which is passed through the first argument register `rdi`).



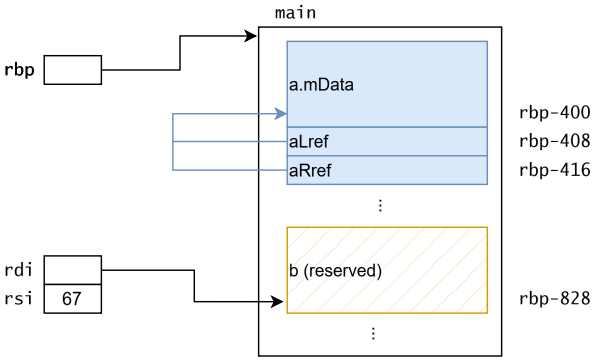
Construction

Given a pointer to a chunk of memory, a *constructor* is a function that initializes (starts lifetime of) an object at that location.
It should be implemented such that the resulting object satisfies the *class invariants*.

Nit: lifetimes may start without any construction/initialization (namely implicit-lifetime types).



Analyzing Construction




```
lea     rdi, [rbp - 828]
mov     esi, 67
call    Gadget::Gadget(int) [base object constructor]
```

New Expressions

New expressions are a language feature that allows for dynamic allocation and initialization of objects, in a simply and customizable way. A new expression looks up and calls the function `operator new()` (this is the customization point).

A *new expression* looks like `new (newArg1, newArg2) Gadget(ctorArg1, ctorArg2)`. When the compiler sees this, it performs a lookup for `operator new(size_t, ...)` which returns a pointer. It then calls the constructor of Gadget on the pointer.



New Expressions

```
/*
// automatically defined defaults, without having to include any headers!
// these use malloc and free under the hood
void* operator new(std::size_t) {...}
... operator new[], operator delete, etc
*/

{
    // mallocs and constructs a Gadget on the heap
    auto x = new (newArg1, newArg2) Gadget(ctorArg1, ctorArg2);
}

{
    // mallocs space for a Gadget
    auto* tmp = static_cast<Gadget*>(operator new(sizeof(Gadget), newArg1, newArg2));
    // pseudocode: call Gadget constructor on the malloc result
    Gadget::Gadget(tmp, ctorArg1, ctorArg2);
    auto x = tmp;
}
```



Operator New (Placement New Overload)

The language does not let you call `Gadget::Gadget(...)` on any arbitrary chunk of memory and initialize an object there.

> *Placement new* is a workaround (uses header `<new>`) that allows you to call the constructor on any given address.

```
new ((void *) 123456) Gadget{67};

// gets transformed into approximately this pseudocode
{
    // this operator new overload just returns 123456
    auto* tmp = static_cast<Gadget*>(operator new(sizeof(Gadget), 123456));
    // thus the constuctor is called on tmp
    Gadget::Gadget(tmp, 67);
}
```



Destruction

A destructor is a function that ends the lifetime of an object. It should be implemented such that it performs any necessary cleanup (e.g. freeing owned resources).

For every initialization of an object that does not have *dynamic storage duration* (i.e. not initialized via operator new), the compiler generates a call to this object's destructor at the appropriate location.
All objects with dynamic storage duration must be destroyed manually.



Destructor Generation Example

```
// what you write
void f(bool flag) {
    auto a = Gadget{1};
    {
        auto b = Gadget{2};
        auto c = Gadget{3};
    }
    if (flag) {
        auto d = Gadget{4};
        return;
    }
    auto e = Gadget{5};
}
```

Destructor calls are generated at every scope exit, in *reverse order of construction*.

```
// what the compiler generates (pseudocode)
void f(bool flag) {
    Gadget::Gadget(&a, 1);
    {
        Gadget::Gadget(&b, 2);
        Gadget::Gadget(&c, 3);
        Gadget::~Gadget(&c);
        Gadget::~Gadget(&b);
    }
    if (flag) {
        Gadget::Gadget(&d, 4);
        Gadget::~Gadget(&d);
        Gadget::~Gadget(&a);
        return;
    }
    Gadget::Gadget(&e, 5);
    Gadget::~Gadget(&e);
    Gadget::~Gadget(&a);
}
```



Storage Re-use

Ending an object's lifetime does not release its storage.
We are free to construct a different object in that storage.

```
{
    auto g = Gadget{};  // lifetime of g begins
    g.~Gadget();        // lifetime of g ends, but its storage is still there

    auto* w = new (&g) Widget{}; // construct widget where g previously was
} // g.~Gadget() called here (UNDEFINED BEHAVIOR)
```



Class Design Conventions

- There are a bunch of *conventions* to follow when implementing a class.
- Most programmers will assume these conventions hold when reading code.
- Most templated libraries (e.g. std::vector<T>) will assume that the type `T` you supply it obeys some of these conventions. Non-compliant types will lead to bugs.

A well designed class satisfies the following *conventions*:

- Design by contract (class invariants, preconditions, postconditions)
- RAII and ownership semantics (acquire resources in ctor, release in dtor)
- Copy semantics (copy constructor creates an independent duplicate)
- Move semantics (move constructor transfers ownership of resources)



Class Invariants

An object is just a bit string, viewed through the lens of a type.
Some bit strings are not valid objects (not well-formed).
E.g. a vector with mSize $>$ mCapacity is nonsensical.

The rules that must be satisfied for an object to be well-formed are called the *class invariants*. When designing a class, the programmer decides what these invariants should be.

For a well designed class, a public method call on a well-formed object should always leave it in a well-formed state.

These ideas are explored more deeply in Elements of Programming (software/philosphy book).



Ownership

An object *owns* something if it is responsible for destroying it. The language enforces this only for subobjects: members (and bases) die when the enclosing object dies.

If an object contains a pointer to another object, this pointer is either owning or non-owning. The compiler does not know or check who owns what. Ownership should be documented (as should class invariants, etc.).

If a class *owns* a resource, then it should do so according to RAII.



RAII

Resource Acquisition is Initialization (RAII)

The name of a C++ programming technique/pattern/idiom which binds the life cycle of a resource to the lifetime of an object. It is done by acquiring the resource in the object's constructor, and releasing it in the destructor.

```
// Non-RAII design
struct IntBox {
    int *mData;
    void acquire(int x) { mData = new int(x); }
    void release() { delete mData; }
};
```

```
// RAII design
struct IntBox {
    int *mData;
    IntBox(int x) : mData{new int(x)} {}
    ~IntBox() {delete mData;}
};
```



Copy Construction

A copy constructor is a constructor that takes an existing object of the same class as its argument.

Defining a copy constructor allows code like this to compile:

```
/* 1.01 */
auto a = Gadget{};
auto& aLref = a;
auto&& aRref = std::move(a);

{
    // all these are copy constructions
    auto c = Gadget{a};
    auto d = Gadget{aLref};
    auto e = Gadget{aRref};
    // this is copy assignment, not copy construction
    e = a;
}
```



Copy Construction Convention

In a well-designed class, copy construction should create a "fully independent duplicate", such that both objects can be used and destroyed independently.

The compiler auto-generates a copy-ctor that recursively copies each member.

```
struct BadIntBox {
  int *mData;
  BadIntBox(int x) : mData{new int(x)} {}
  ~BadIntBox() { delete mData; }
  // auto-generated copy constructor
  BadIntBox(const BadIntBox& other) : mData{other.mData} {}
};
```

Usually, automatically generated copy constructors are NOT well-designed.
In this example, the resultant copy cannot be used nor destroyed independently.



Double Frees

Double frees occur when you write buggy classes that don't handle copying properly.
Consider the below example using the `BadIntBox` we developed earlier

```
int main() {
    auto original = BadIntBox{42};
    auto copy = BadIntBox{original}; // copy.mData = original.mData (both point to same
    ↪  heap object!)

    // ~original; ~copy; -> Double Free!
}
```

When the destructors run, both objects think that they are responsible for destroying the heap-allocated integer and freeing it. This results in Undefined Behavior!
To fix this, think carefully when implementing your copy constructors.
Make your class non-copyable if applicable.



Copy Constructor Impl

To perform a proper *deep copy* on our `IntBox`, we want

- a new region of memory to be allocated on the heap
- the `other.mData` integer is copy constructed into this new region

```
struct IntBox {
  int *mData;
  IntBox(int x) : mData{new int(x)} {}
  ~IntBox() { delete mData; }
  IntBox(const IntBox& other) : mData{new int(*(other.mData))} {}
};
```



Value Categories

Every *expression* has a *value category* (lvalue or rvalue).

```
struct Lorem { struct Ipsum { int x; } y; int z; } l;
auto* ptr = &l.z;
int arr[3] = {1, 2, 3};
```

lvalues — has identity (an address)

```
l             // named variable
l.z           // member access
l.y.x         // nested member
*ptr          // dereference
arr[0]        // subscript
```

Can appear on the *left* side of assignment.
Can take the address with &.

rvalues — no persistent identity

```
42            // literal
l.z + 1       // arithmetic result
IntBox(3)     // temporary object
std::move(l)  // cast to rvalue
int{}         // value-initialized tmp
```

Cannot appear on the left side of assignment.
Cannot take the address with &.



rvalue references

An *rvalue reference* is a reference that can bind to an *rvalue*.

When a ref is bound to a tmp, lifetime extended until the ref goes out of scope.
i.e. the compiler postpones the dtor to the end of the scope.

```
/* 2.01 */
{
    // tell compiler: we want to extend the tmp's lifetime until end of scope
    IntBox&& rref = IntBox(3);
    // play around with the temporary via the rvalue ref
    std::cout << rref.get() << std::endl;
    rref.get() *= 10;
    std::cout << rref.get() << std::endl;
}
```



Arguments by rvalue ref

We already know that a function can take arguments by (lvalue) reference.
It is also possible for a function to take arguments by rvalue reference.

```
/* 2.01 */
void foo(IntBox& obj) { obj.mData += 42; }
void foo(IntBox&& obj) { obj.mData = nullptr; }

int main() {
    // If you pass in an rvalue (e.g. the tmp result of the ctor function IntBox(3)),
    // the rvalue ref extends the lifetime of the tmp until the function returns.
    foo(IntBox(3));
}
```



Reference Binding

```
/* 2.01 */
{
    IntBox{3};                      // dies here
    auto&& rref = IntBox{3};        // extended
    const auto& lref = IntBox{3};   // extended
    auto x = IntBox{3};             // no tmp
}   // x, lref, rref destroyed here
{
    auto& data = IntBox{3}.get();  // UB!
    std::cout << data << std::endl;
}
```

Output

```
constructor
destructor
constructor
constructor
constructor
destructor
destructor
destructor
constructor
destructor
894751456
```

`auto&&` and `const auto&` extend a tmp's lifetime, `auto&` cannot bind to tmps. Only *direct* binding extends: `get()` returns a ref to an object that dies immediately.



Overload resolution

Given that an argument has two overloads, one taking lvalues and one taking rvalues, how does the compiler determine which overload to dispatch for each call-site?

```
{
    auto x = IntBox{3};
    IntBox&& rref = std::move(x);

    foo(x); // calls lvalue overload
    foo(rref); // also calls lvalue overload (weird)
    foo(IntBox(3)); // calls rvalue overload
    foo(std::move(x)); // calls rvalue overload
    foo(std::move(rref)); // calls rvalue overload
}
```

The compiler picks the overload depending on the *value category* of the argument.



Move Construction

A move constructor is a constructor that takes an rvalue reference to an object of the same type.

```
struct IntBox {
    // ctor
    IntBox(int x) : mData{new int(x)} {}
    // copy ctor
    IntBox(const IntBox& other) : mData{new int(*(other.mData))} {}
    // move ctor
    IntBox(IntBox&& other) : mData{std::exchange(other.mData, nullptr)} {}
};

int main() {
    auto original = IntBox{42};
    auto copied = IntBox{original};
    auto moved = IntBox{static_cast<IntBox&&>(original)};
}
```



Move Construction Convention

In a well-designed class, move ctor should transfer ownership of resources from `other` to `this`, such that upon completion, `this` represents the same abstract entity that `other` used to, whilst `other` should end up in a well-formed null-like state, such that its dtor may be called without issue.

```
struct IntBox {
    // poorly designed move constructor, absolute nonsense
    IntBox(IntBox&& other) : mData{(int*) 12345} {}

    // poorly designed move constructor, shares ownership, double-free and UB
    IntBox(IntBox&& other) : mData{other.mData} {}

    // poorly designed move constructor, transfers ownership
    // but `other` is no longer well-formed. UB upon destruction.
    IntBox(IntBox&& other) : mData{std::exchange(other.mData, (int*) 12345)} {}
};
```



Returning structs from functions

In the following code example we have a copyable `IntBox` class.

```
/* 3.02 */
IntBox sum(IntBox& a, IntBox& b) {
    auto result = *(a.mData) + *(b.mData);
    return IntBox(result);
}
IntBox product(IntBox& a, IntBox& b) {
    auto result = *(a.mData) * *(b.mData);
    auto retval = IntBox{result};
    return retval;
}
int demo() {
  auto a = IntBox{5}; auto b = IntBox{6};
  auto sumResult = sum(a, b);
  auto prodResult = product(a, b);
}
```



Returning structs from functions

In the stack frame of the caller (`demo()`), the compiler allocates a *return_slot*. The address of the return slot is implicitly passed as an argument.

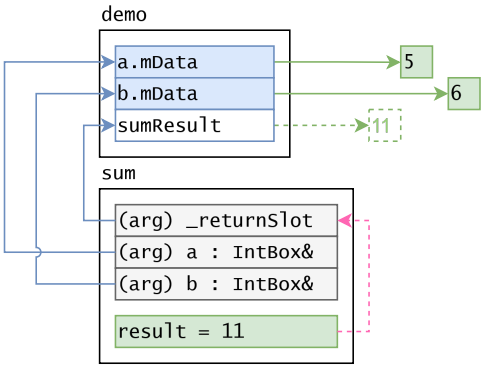
- if the returned expr is a local variable, it gets copy-ctored into the return slot.
- if the returned expr is a ctor call, the ctor is run directly on the return slot.

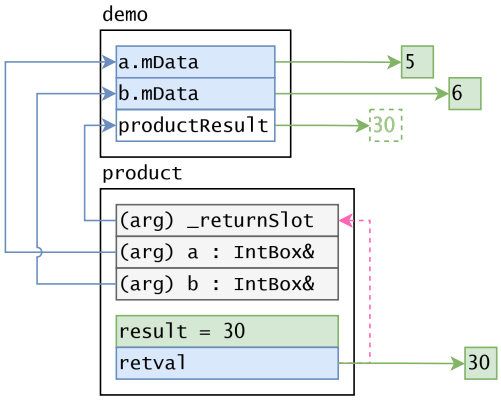
Conceptually, this is what happens under the hood:

```
/* 3.01 */
auto a = IntBox{5}; auto b = IntBox{6};
alignas(IntBox) uint8_t _storageSumResult[sizeof(IntBox)]; // allocate return slot on stack
// inlined function
{   auto* _returnSlot = reinterpret_cast<IntBox *>(_storageSumResult);
    auto& argA = a; auto& argB = b; // argument binding
{
    auto result = *(argA.mData) + *(argB.mData);
    new (_returnSlot) IntBox(result); // RVO: constructor runs directly in return slot
}}
```



Returning structs from functions





Temporaries

If the caller doesn't bind the return value to a variable, it still allocates a return slot on the stack.
Methods may be called on this temporary while within the full expression.
However immediately following the full expression, the temporary is destroyed.

```
/* 3.01 */
int demo() {
    // return slot is to a temporary, rather than an IntBox object
    std::cout << product(a, b).get() << std::endl;
    // _tmp.~IntBox() // the compiler generates a call to the destructor here
}
```

It is possible to bind the temporary to an r-value reference (later section), for example by passing it into a function that takes an r-value reference as its argument.



Issue: Unnecessary Copies

Our `IntBox` from 3.01 is copyable, but has no move constructor.

```
/* 3.01 */
IntBox product(IntBox& a, IntBox& b) {
    auto result = *(a.mData) * *(b.mData);
    auto retval = IntBox{result}; // allocates an int
    return retval;                // copy ctor: allocates another int
}                                 // ~retval(): frees the first int

int demo() {
    auto a = IntBox{5}; auto b = IntBox{6};
    auto c = product(a, b);       // c is the return slot
}
```

Without RVO, `retval` is copy constructed into the return slot, then immediately destroyed. We pay for an extra allocation and copy, only to throw the original away.

`retval` is about to die anyway. Can we steal its resources instead of copying them?



Solution: Define a Move Constructor

```
IntBox(IntBox&& other) : mData{std::exchange(other.mData, nullptr)} {}
IntBox product(IntBox& a, IntBox& b) {
    auto result = *(a.mData) * *(b.mData);
    auto retval = IntBox{result};
    return retval; // move construct into returnSlot: no allocation, very fast
}                  // ~retval(): destroys moved-from object
```

A local returned by name is treated as an rvalue. The copy ctor was previously called, because const lrefs can also bind to rvalues.

Don't write `return std::move(retval);`, as this disables NRVO.



Motivation: MaybeInt

Suppose we want a type that either holds a value, or nothing (like a nullable int):

```
struct MaybeInt {
    bool mHasValue; int mValue;

    MaybeInt() : mHasValue(false), mValue{} {}
    MaybeInt(int v) : mHasValue(true), mValue(v) {}

    bool has() const { return mHasValue; }
    int get() const {
        if (!mHasValue) throw std::runtime_error("empty");
        return mValue;
    }
};
```

What if we want `MaybeDouble`? `MaybeString`? Same logic, different type.



Maybe - Templated Solution

We can generate the exact same code by writing a *template class*.

```
/* 4.02 */
template <typename T>
struct Maybe {
    bool mHasValue; T mValue;
    /* ... */
};
```

Verify this with `cppinsights.io`.



Template Instantiation

```
  /* 4.02 - CPPINSIGHTS OUTPUT */

#ifdef INSIGHTS_USE_TEMPLATE
template<> struct Maybe<int> {
  bool mHasValue;
  int mValue;
  /* ... */
  inline const int & get() // ...
};
#endif

#ifdef INSIGHTS_USE_TEMPLATE
template<> struct Maybe<double> {
  bool mHasValue;
  double mValue;
  /* ... */
  inline const double & get() //...
};
#endif
```

For each type (`int`, `std::string`, `double`) that is substituted for the template type parameter `T`, an *instantiation* is generated.
Based on the code, the compiler determines which instantiations are necessary, and will only generate those which are used.

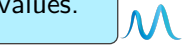


C++ Templates

```
// declare Widget as a template class
template <typename T, typename U, int V> struct Widget{/* */};
// declare void foo(T,U) as a template function
template <typename T, typename U> void foo(T lorem, U ipsum) {/* */}

int main() {
    auto s = std::string{"hi"};
    auto val = 42;
    foo(s, val); // instantiation <T = std::string, U = int>
    auto obj = Widget<int, decltype(s), 20>{}; // instantiation <T = int, U = decltype(s), V
    ↪  = 20>
    auto obj2 = Widget<int, int, val>{}; // COMPILE ERROR: val is not a compile-time value
}
```

The programmer can *use* the template entity (function / class). At compile-time, the compiler *stamps* out every required instantiation, replacing each template parameter with the required types / compile-time values.



What can be templated?

Templates can be used to define:

- **Function templates**: generate function definitions, with types substituted
- **Class templates**: generate class definitions, with types substituted.
- **Variable templates**: generate variable definitions, with types substituted

Let's take a look at some more examples.



Example: `max()` without function templates

Suppose we are writing our own custom `cmath` library:

```
int max(int x, int y) {
    return (x > y) ? x : y;
}
double max(double x, double y) {
    return (x > y) ? x : y;
}
char max(char x, char y) {
    return (x > y) ? x : y;
}

int main() {
    std::cout << max(1, 2) << '\n';     // overload resolution: max(int, int)
    std::cout << max(3.5, 7.5) << '\n'; // overload resolution: max(double, double)
    std::cout << max('a', 'b') << '\n'; // overload resolution: max(char, char)
    return 0;
}
```

Can we do better?



Example: `max()` with function templates

```cpp
#include <iostream>

template <typename T>
T max(T x, T y) {
    return (x > y) ? x : y;
}

int main() {
    std::cout << max(1, 2) << '\n';     // deduces to max<int>(int)
    std::cout << max(3.5, 7.5) << '\n'; // deduces to max<double>(double)
    std::cout << max('a', 'b') << '\n'; // deduces to max<char>(char)
    return 0;
}
```

What about `max(1, 2.1)`?



Example: Multiple type parameters

```
#include <iostream>

template <typename T, typename U>
auto max(T x, U y) {
    return (x > y) ? x : y;
}

int main() {
    std::cout << max(1, 2.1) << '\n'; // 2.1
    return 0;
}
```

Using `auto`, the compiler deduces the return type.



Where have we seen function templates?

`std::sort` from `<algorithm>` works on any container of any type:

```
#include <algorithm>
#include <vector>

auto ints = std::vector<int>{3, 1, 2};
auto doubles = std::vector<double>{3.5, 1.1, 2.2};
auto strings = std::vector<std::string>{"banana", "apple", "cherry"};

std::sort(ints.begin(), ints.end());
std::sort(doubles.begin(), doubles.end());
std::sort(strings.begin(), strings.end());
```

Under the hood, `std::sort` is a function template; same code, "stamped out" once for each type it is called with.



Template instantiation examples

```
/* 4.03 */
// an array of 8 integers
struct ArrayInt8 {
    using T = int;
    static constexpr std::size_t N = 8;
    T mData[N];
    // set array element to a value by index
    void set(std::size_t index, T val) {/* */}
    // get const reference to array element by index
    const T& at(std::size_t index) const {/* */}
};

// returns two values: {maximum in nums, average of nums}
Pair<int, double> getMaxAndAverage(const ArrayInt8& nums) {
    /* ... */
    return {currMax, currSum / 8};
}
```

Copy the full code example into cppinsights to see all generated instantiations.



Templating the array class

```
/* 4.04 */
template <typename T, std::size_t N>
struct Array {
    T mData[N];
    // set array element to a value by index
    void set(std::size_t index, T val) {/* */}
    // get const reference to array element by index
    const T& at(std::size_t index) const {/* */}
};

// returns two values: {maximum in nums, average of nums}
Pair<int, double> getMaxAndAverage(const Array<int, 8>& nums) {
    /* ... */
    return {currMax, currSum / 8};
}
```

Copy the full code example into cppinsights to see all generated instantiations.



Templating the function

```
/* 4.05 */
template <typename T, std::size_t N>
// requires: T - convertible to double
Pair<T, double> getMaxAndAverage(const Array<T, N>& nums) {
    auto currMax = nums.at(0);
    auto currSum = 0.0;
    for (std::size_t i = 0; i < N; ++i) {
        currMax = currMax > nums.at(i) ? currMax : nums.at(i);
        currSum += nums.at(i);
    }
    return {currMax, currSum / N};
}
```

Copy the full code example into cppinsights to see all generated instantiations.



Instantiation Failures

```
/* 4.06 */
template <typename Container, typename Value>
Value pop(Container& c) {
    c.pop_front();
    /* ... */
}

/* compiler generated instantiation */ /*Container = double, Value = int*/
int pop<double, int>(double& c) {
    c.pop_front(); // COMPILE ERROR!
    /* ... */
}

int main() {
    auto x = double{};
    pop(x, 1);
}
```



Exercise: Queue

You are given an implementation of a queue of `int`.
Make it generic.

```
/* 4.08 */
// given a queue of ints
class Queue {/* ... */};

// implement a queue of "any" type
template <typename T>
class Queue {/* ... */};
```



Bounded Vector

When a caller passes an rvalue into a function, by convention, this means that the caller is giving permission for the callee to *move construct* from the ref.

In this code example we define a vector class with a fixed capacity.

```
/* 5.01 */
template <typename T, std::size_t Capacity>
struct BoundedVector {
    // lifetime and resource management
    BoundedVector() {/* */}
    ~BoundedVector() {/* */}
    BoundedVector(const BoundedVector<T, Capacity>& other) {/* */}
    // member functions
    void pushBack(T& obj) {/* */}
    void pushBack(T&& obj) {/* */}
};
```



Why do we want the rvalue overload?

What would happen if we didn't have this rvalue overload for `pushBack`?

```
int main() {
    auto vec = BoundedVector<IntBox, 8>{};
    vec.pushBack(IntBox(3)); // COMPILE ERROR

    // required either one of these to be defined:
    // - pushBack(const IntBox& obj) [because const lvalue refs can bind to rvalues]
    // - pushBack(IntBox&& obj)
}
```



Why do we want the rvalue overload?

What would happen if we made `pushBack` take a *const* ref instead?

```
/* 5.01 */
template <typename T, std::size_t Capacity>
struct BoundedVector {
    /* ... */
    void pushBack(const T& obj) {
        // this function is NOT permitted to move from obj
        // since it is a const reference
    }
};

int main() {
    auto vec = BoundedVector<IntBox, 8>{};
    vec.pushBack(IntBox(3));
    // copy constructs into the vector
    // this may be expensive if type T handled more resources
}
```



The issue: repeated logic

For this particular use case (`pushBack()`), we run the same logic when both rvalues and lvalues are passed as arguments. We just want to *forward the argument perfectly* into the constructor.

```
/* 5.01 - BoundedVector */
void pushBack(T& obj) {
    assert(mSize < Capacity);
    new (mData + mSize) T(obj);              // calls constructor with obj (lvalue)
    mSize++;
}

void pushBack(T&& obj) {
    assert(mSize < Capacity);
    new (mData + mSize) T(std::move(obj));  // calls constructor with obj (rvalue)
    mSize++;
}

// note that even though decltype of obj is T&&
// the expression "obj" is still lvalue category, so we must std::move to make rvalue
```



Invalid solution: attempt to deduce `F = T&`

```
template <typename F>                                  // e.g. hope that: F = T&&
void pushBack(F obj) {                                // e.g. hope that: T&& obj
    assert(mSize < Capacity);
    new (mData + mSize) T(static_cast<F>(obj));       // e.g. static_cast<T&&>(obj)
    mSize++;
}
/* ... */
vec.pushBack(IntBox(3));
```

The idea is that we hope `F` can deduce to either `T&` or `T&&`, depending on what value category we use at the call-site.

Template argument deduction will strip references if the argument is declared as `(F obj)`. i.e. `F` will always deduce to just `T`, not `T&` or `T&&`.



Bad solution: explicit instantiation

```
template <typename F>                                  // e.g. F = T&&
void pushBack(F obj) {                                // e.g. T&& obj
    assert(mSize < Capacity);
    new (mData + mSize) T(static_cast<F>(obj));       // e.g. static_cast<T&&>(obj)
    mSize++;
}
/* ... */
vec.pushBack<IntBox &&>(IntBox(3));
```

We have gotten rid of duplicate logic, at the cost of needing each call-site to explicitly specify the template parameter. If you miss one, it silently makes a by-value overload (which is the worst thing ever).



The solution: universal references

The *universal/forwarding reference* technique is when a function parameter has declared type `F&&`, where `F` is a template argument of the function.

```
/* 5.02 - BoundedVector */
template <typename F>
void pushBack(F&& obj) {
    assert(mSize < Capacity);
    new (mData + mSize) T(static_cast<F&&>(obj));
    mSize++;
}
```

At first glance, wouldn't this only produce an rvalue overload?



How does it work

There is no special compiler logic that handles universal references.
Two parts of the C++ language come together to make universal references possible:

- Reference collapsing rules
- A special rule for template argument deduction (next slide)

Ref collapsing rules determine the resultant type when you take a ref to a ref.

```
using L = int&;     // int &      -> int&
  using LL = L&;    // int & &    -> int&
  using LR = L&&;   // int & &&   -> int&   (relevant to universal refs)

using R = int&&;    // int &&     -> int&&  (relevant to universal refs)
  using RL = R&;    // int && &   -> int&
  using RR = R&&;   // int && &&  -> int&&
```



Template argument deduction rule

When a function parameter has form F, where T is a template parameter being deduced, and the argument is an lvalue of type A, then T is deduced as A.

```
template <typename F>
void pushBack(F&& obj) { // function parameter has form F&&
    /* ... */
}

auto x = Widget{5};
v.pushBack(x);
// argument is an lvalue of type Widget
// thus F is deduced as Widget&

v.pushBack(std::move(x))
// argument is an rvalue of type Widget
// F is deduced as Widget
```



How does it work

When called with both value categories, two instantiations are created:

```
template <> // F = T
void pushBack(T && obj) {
    assert(mSize < Capacity);
    new (mData + mSize) T(static_cast<T &&>(obj));
    mSize++;
}

template <> // F = T&
void pushBack(T& && obj) {                          // F&&  -> T & && -> T&
    assert(mSize < Capacity);
    new (mData + mSize) T(static_cast<T& &&>(obj));  // F&&  -> T & && -> T&
    mSize++;
}
```

`std::forward<F>` is defined as `static_cast<F&&>`.

